

Dolly POS — Technical Handoff Document

Engineering Architecture, Codebase Walkthrough, Database ERD & Development Changelog

1. Technology Stack & Pinned Versions

Layer	Technology / Framework	Version	Role & Purpose
Backend Framework	Python FastAPI	0.141.1	High-speed async REST API core
ASGI Web Server	Uvicorn (Standard)	0.52.4	High-concurrency async HTTP server
ORM / DB Engine	SQLAlchemy + Psycopg2	2.0.52 / 2.9.12	Relational data mapping & connection pooling
Database	PostgreSQL (x64)	16.x	ACID transactional enterprise database
Frontend Library	React 18 + TypeScript	18.2.0 / 5.2.2	Type-safe interactive single page application
Build Tool & Bundler	Vite	5.1.6	Ultra-fast HMR and optimized ES build
UI Styling & Icons	Tailwind CSS + Lucide	3.4.1 / 0.363	Modern responsive glassmorphic design system
State Management	Zustand	4.5.2	Persistent lightweight client state (Auth & Cart)
PDF Generation	ReportLab	5.0.1	Programmatic invoice & manual PDF engine
Data Analytics	Pandas + OpenPyXL	3.0.5 / 3.1.5	Excel export and financial aggregation

2. Project Directory Structure & Manifest

Root Folder: c:\Users\SomeshBang\Desktop\Antigravity DollyPos\

Antigravity DollyPos/ ■■■ DollyPOS_Launcher.bat # Root one-click Windows launcher ■■■ LICENSE # MIT Open Source License ■■■ README.md # Complete technical handoff mirror ■■■ backend/ # FastAPI Python Core Backend ■ ■■■ desktop_app.py # Production standalone desktop server runner ■ ■■■ requirements.txt # Pinned Python package dependencies ■ ■■■ .env.example # Clean configuration template ■ ■■■ app/ ■ ■ ■■■ config.py # Global settings, DB connection string, JWT secret ■ ■ ■■■ main.py # FastAPI bootstrap, lifespan, static mount & CORS ■ ■ ■■■ core/ ■ ■ ■■■ database.py # SQLAlchemy SessionLocal & connection engine ■ ■ ■■■ security.py # Password hashing (bcrypt) & JWT token encoders ■ ■ ■■■ models/ # SQLAlchemy Relational Models (12 tables) ■ ■ ■■■ user.py, product.py, category.py, invoice.py, customer.py ■ ■ ■■■ vendor.py, purchase.py, expense.py, settings.py, audit_log.py ■ ■ ■■■ return_order.py, lost_demand.py, whatsapp_log.py ■ ■ ■■■ api/ # REST API Endpoint Routers ■ ■ ■■■ auth_router.py, product_router.py, invoice_router.py ■ ■ ■■■ customer_router.py, vendor_router.py, purchase_router.py ■ ■ ■■■ returns_router.py, expense_router.py, reports_router.py ■ ■ ■■■ settings_router.py, backup_router.py, marketing_router.py ■ ■ ■■■ services/ # Core Business Logic Services ■ ■ ■■■ analytics_service.py # Dashboard KPIs, 5Y smooth cubic spline curves ■ ■ ■■■ printer_service.py # Raw TSPL barcode & ESC/POS receipt generation ■ ■ ■■■ whatsapp_service.py # Cloud API messaging & birthday cron ■■■ frontend/ # React 18 + TypeScript + Vite SPA ■ ■■■ package.json # Pinned Node dependencies & npm scripts ■ ■■■ vite.config.ts # Vite bundler configuration & proxy ■ ■■■ src/ ■ ■ ■■■ App.tsx, main.tsx # React Root router & application shell ■ ■ ■■■ pages/ # Main Page Views ■ ■ ■■■ DashboardPage.tsx # Live KPIs, smooth cubic Bézier 5Y chart ■ ■ ■■■ BillingPage.tsx # Barcode checkout, split payment, thermal bill ■ ■ ■■■ InventoryPage.tsx # Stock management, 1-Up/2-Up barcode printer ■ ■ ■■■ CustomersPage.tsx # Khata ledger, Udhar WhatsApp recovery ■ ■ ■■■ VendorsPage.tsx # Wholesale suppliers, purchase inward orders ■ ■ ■■■ ReportsPage.tsx # P&L; accounting, YoY comparison, Excel export ■ ■ ■■■ SettingsPage.tsx # Printers, sound toggle, master key, backup ■ ■ ■■■ LoginPage.tsx # Dual-role authentication gate ■ ■ ■■■ components/ # Modular UI Components (Navbar,

```
Modals, Tables) ■ ■ ■■■ store/authStore.ts # Zustand persistent JWT token store ■■■ docs/ #
Publication-grade PDF Documentation ■ ■■■ Dolly_POS_User_Manual.pdf ■ ■■■
Dolly_POS_Technical_Handoff.pdf ■ ■■■ Dolly_POS_Disaster_Recovery_and_New_Laptop_Guide.pdf ■■■
installer/ # Windows Installation & Uninstallation Scripts ■■■ Install-DollyPOS.bat ■■■
Uninstall-DollyPOS.bat ■■■ setup_inno.iss
```

3. Deep Code Walkthrough & Interconnections

- **app.core.database (database.py)**

Manages SQLAlchemy engine with PostgreSQL connection pool (pool_size=20, max_overflow=10). Yields thread-safe SessionLocal instances for FastAPI dependency injection via get_db().

- **app.services.analytics_service (analytics_service.py)**

Executes sub-10ms SQL aggregations for Dashboard metrics. Implements smart timeline granularity: Daily (<=45d), Weekly (46-400d), and Monthly (>400d/5Y). Returns smooth timeline curves, 50/30 category balancing metrics, and Units Per Transaction (UPT) cross-sell KPIs.

- **app.services.printer_service (printer_service.py)**

Generates raw TSPL (TSC Printer Language) byte streams for 1-Up (50x25mm) and 2-Up (100x50mm) barcode thermal printing. Generates 80mm ESC/POS receipt commands with store logo, QR code, and thermal cut pulses.

- **app.api.invoice_router (invoice_router.py)**

Handles billing transactions: validates barcode stock, calculates taxes, applies item/bill discounts, creates immutable Invoice and InvoiceItem records, reduces product stock quantity, logs payment method (Cash, UPI, Split, Khata), and triggers WhatsApp digital receipt dispatch.

- **app.api.backup_router (backup_router.py)**

Implements full relational database dump and restoration to/from self-contained JSON backup files. Strips sensitive machine-specific store credentials during export and validates foreign keys during import.

- **frontend.src.pages.DashboardPage (DashboardPage.tsx)**

Renders real-time retail pulse, KPI cards (Today's Sales, Monthly Revenue, Today's Net Profit, Inventory Valuation), Cash Flow Liquidity strip, and interactive SVG Monotone Cubic Bézier Spline revenue curves with dynamic hover HUD.

- **frontend.src.pages.BillingPage (BillingPage.tsx)**

High-speed counter checkout terminal. Features barcode laser listening, speed dial quick selection, live cart subtotals, split payments modal, change due calculator, and direct thermal print triggers.

4. Database Schema & Relational Structure

Table Name	Primary Key	Foreign Keys	Indexed Columns & Constraints
users	id (Int)	None	username (Unique), role, is_active
categories	id (Int)	None	name (Unique)
subcategories	id (Int)	category_id -> categories.id	category_id, name
products	id (Int)	category_id, subcategory_id	barcode (Unique), sku, speed_dial, is_active
invoices	id (Int)	customer_id, user_id	bill_number (Unique), created_at, payment_mode
invoice_items	id (Int)	invoice_id, product_id	invoice_id, product_id
customers	id (Int)	None	phone (Unique), name, current_balance
customer_ledger	id (Int)	customer_id, invoice_id	customer_id, transaction_date
purchases	id (Int)	vendor_id	vendor_id, purchase_date, invoice_number
expenses	id (Int)	None	expense_date, category

5. Historical Bug Log & Resolutions

• Bug: Database Restore Foreign Key Violation (purchases / purchase_items) (backend/app/api/backup_router.py)

Root Cause: Deleting products during database restore violated foreign keys on purchase_items and return_items.

Fix Applied: Added ON DELETE CASCADE to purchase_items and return_items foreign keys, and wrapped restore in sequential topological deletion.

• Bug: Dashboard Net Profit Today showing negative value (backend/app/services/analytics_service.py)

Root Cause: Expenses query in get_dashboard_summary was not bounded to today, subtracting entire lifetime expenses.

Fix Applied: Strictly bounded Expense.expense_date between today_start (00:00:00) and today_end (23:59:59).

• Bug: 5-Year Timeline Chart Clutter (1,825 daily jagged spikes) (frontend/src/pages/DashboardPage.tsx)

Root Cause: Plotting daily points over 5 years created dense vertical line spikes.

Fix Applied: Implemented Smart Monthly Grouping (61 months) in backend and Monotone Cubic Bézier Spline paths in frontend SVG.

• Bug: Damaged Stock Product Search dropdown staying open (frontend/src/pages/DamagedStockPage.tsx)

Root Cause: Dropdown had no onBlur / click-outside handler.

Fix Applied: Added active input focus tracking and click-outside dismissal refs.

• Bug: Footer Settings Desynchronization between Settings and Printout (frontend/src/pages/SettingsPage.tsx)

Root Cause: Settings saved into browser localStorage while receipt generation read from database.

Fix Applied: Migrated all footer styling (font size, bold, terms) to database table store_settings with live sync.